Section A

Q1.

Ans:- Client-Server Model

When a customer clicks the “View Menu” button, the following steps take place:

1. Browser (Client): The browser sends an HTTP request to the server to retrieve the restaurant menu.
2. DNS (Domain Name System): DNS converts the website's domain name into an IP address so the browser can locate the correct server.
3. Web Server: The web server receives the request and queries the database to retrieve the restaurant's menu details.
4. HTTP Response: The server sends the menu data back to the browser through an HTTP response, and the browser displays the menu to the customer.
5. Caching: On a second visit, the menu may load faster because the browser can reuse cached resources instead of downloading them again. This behaviour is known as client-side caching.

Request flow:

Browser -> DNS -> Web Server -> Database -> HTTP Response -> Browser

Q2.

Ans:- For the food delivery checkout page, I would use the following HTML form elements:

1. Multi-line delivery address — <textarea>

The <textarea> element allows customers to enter their complete delivery address across multiple lines, including house number, street, and area.

2. Payment method — <select> and <option>

The <select> element creates a dropdown menu, while <option> elements provide the three choices: Cash, Card, and UPI. This ensures customers select one of the predefined payment methods.

3. Save address checkbox — <input type="checkbox">

A checkbox allows customers to select or deselect the option to save their delivery address for future orders.

Using a plain text input for all three fields would be a poor decision because it does not support multi-line address entry, does not provide a predefined dropdown of payment methods, and does not provide a selectable checkbox. Using the correct elements improves usability, accessibility, and data collection.

Q3.

Ans:- The appropriate HTML5 semantic elements for the four areas are:

1. Top navigation — <nav>: Used to contain navigation links such as Home, Restaurants, and Contact.
2. Hero banner — <section>: Used to group the main banner content, such as a heading, description, and Order Now button.
3. Restaurant listing — <main> or <section>: <main> identifies the primary content of the page, while <section> groups the restaurant listings under a heading.
4. Footer — <footer>: Used to contain footer information such as copyright details and contact information.

Two consequences of using for everything

1. Screen readers: A is a generic container and does not provide semantic meaning. Using only elements make it harder for screen-reader users to identify and navigate important page regions, such as navigation and the main content.

2. Search engine optimization (SEO): Semantic elements help search engines understand the structure and purpose of website content. Using only elements remove these useful structural signals and may make the content harder for search engines to interpret. However, using elements alone does not automatically lower a website's ranking.

Q4.

Ans:- 1. position: absolute

The position: absolute property positions an element relative to its nearest positioned ancestor. If no such ancestor exists, it is positioned relative to the initial containing block. The element generally scrolls along with the page content.

2. position: fixed

The position: fixed property positions an element relative to the viewport. It remains in the same position on the screen when the user scrolls the page, under normal conditions.

position: fixed is suitable for the cart summary panel because it must remain visible in the top-right corner while the customer scrolls through the restaurant menu.

The top, right, width, and z-index properties position it appropriately, while the main content's right padding helps prevent overlapping.

Q5.

Ans:-

1. Flexbox container properties

I would apply display: flex to the restaurant container to arrange the cards horizontally. The flex-wrap: wrap property allows cards to move to the next row, and gap provides consistent spacing between them.

2. Individual card properties

Each restaurant card would use flex-basis to control its width. On desktop, each card occupies approximately 25% of the available row width, displaying four cards side by side. On tablets with a screen width of 768px or less, each card occupies approximately 50% of the row width, displaying two cards per row.

CSS Grid provides more precise control over both rows and columns. For example, if restaurant cards have different content heights and the design requires consistent alignment across a two-dimensional layout, Grid can control the row and column structure more precisely than Flexbox.

Q6.

Ans:-

1. SASS Variables

A SASS variable allows us to store a reusable value, such as the brand colour #E63946, in one place.

In plain CSS without variables, we would need to manually change the colour in over 50 locations. Using a SASS variable, we can update the colour once, and the new value will be applied wherever that variable is used when the stylesheet is recompiled.

2. SASS Nesting

SASS nesting allows us to write related CSS rules inside a parent selector. For example, the restaurant card's title, image, and Order Now button can be styled inside the .restaurant-card selector.

This makes the code cleaner, more organised, and easier to maintain.

Why is this harder in standard CSS?

Traditional CSS requires separate selectors for each element, making large stylesheets more repetitive and harder to organise. SASS nesting groups related styles together, and variables make global changes easier.

Modern CSS also supports custom properties and native nesting, but SASS provides these features through preprocessing.

Conclusion: SASS variables reduce repeated code and simplify colour changes, while SASS nesting makes the restaurant card's styles more readable, organised, and maintainable.